

Alembic System Design & Implementation Plan

Project: Cloud Backend

Collaborative Design

September 29, 2025

Contents

1	Overview	2
2	Core Architecture: Microservices	2
3	Communication & Data Storage	2
3.1	Inter-Service Communication	2
3.2	Data Storage Strategy	2
4	Key Components & Technology Stack	2

1 Overview

This document outlines the design for the cloud backend, a distributed system of microservices that serves as the central "thinking" component for the Local Agent. It is designed for security, scalability, and high performance, managing all user data, knowledge storage, and complex reasoning tasks.

2 Core Architecture: Microservices

The backend will be a distributed system of independent, collaborating services. This architecture enhances security by compartmentalizing data and functionality.

API Gateway A single, secure entry point for all requests from the Local Agent. It handles request routing, rate limiting, and initial authentication.

Identity Service Manages user and organization profiles, authentication (registration, login), and all sensitive Personally Identifiable Information (PII).

Access Control Service Manages groups, roles, and permissions. It answers authorization queries from other services.

Knowledge Core Service The main application "brain." It manages the lifecycle of *Knowledge Quanta*, including creation, retrieval, and semantic search.

Tooling Service A specialized instance of the knowledge service, dedicated to storing and retrieving knowledge related to tools and workflows.

Audit Service Provides a centralized, immutable logging facility. All other services report critical events to this service for compliance and security tracking.

3 Communication & Data Storage

3.1 Inter-Service Communication

A **Message Bus** (e.g., RabbitMQ, Kafka) will be used for asynchronous communication between services. This decouples the services, ensuring that a failure in one service does not cascade and bring down the entire system.

3.2 Data Storage Strategy

The system will use a "**database per service**" model to ensure strong data encapsulation. This allows for **polyglot persistence**—using the best type of database for each service's specific needs. For example:

- A relational database (e.g., PostgreSQL) for the Identity Service.
- A document or relational database for the Knowledge Core Service's primary data.
- A dedicated Vector Database (e.g., Pinecone, Weaviate) for semantic search.
- An append-only log or time-series database for the Audit Service.

4 Key Components & Technology Stack

Framework **ASP.NET Core** will be used for building fast, secure, and cross-platform web APIs for each microservice.

Containerization **Docker** for packaging each service, and **Kubernetes** for orchestration and scaling in the cloud.

Databases A mix of databases, including PostgreSQL, a dedicated Vector Database, and a message bus like RabbitMQ.

API Gateway A dedicated gateway technology such as **Ocelot** or a cloud-provider solution (e.g., AWS API Gateway).